

Universidad Autónoma de Tamaulipas

Facultad de Ingeniería Tampico



ASIGNATURA

Programación de Interfaces y Puertos

6vo. Semestre – Grupo “I”

2025 -1

TRABAJO

Desarrollo de Tareas e Investigaciones

UNIDAD

4 - Desarrollo e Implementación de Controladores Mediante Interfaces y Puertos

Docente: Dr. García Ruiz Alejandro H.

Integrante del Equipo	Nivel de Participación
Izaguirre Cortes Emmanuel	40%
Garcia Salas Yahir Misael	35%
Turrubiates Mejia Gilberto	25%
Total:	100%

Índice

<i>Índice</i>	<i>1</i>
<i>Tratamiento de datos mediante listas de comprensión.....</i>	<i>2</i>
<i>Modulo Matplotlib</i>	<i>5</i>
<i>Modulo Numpy</i>	<i>11</i>
<i>Instrucción eval en python</i>	<i>16</i>
<i>¿Qué es Adquisición de Datos?.....</i>	<i>18</i>
<i>¿Qué es entorno?.....</i>	<i>23</i>
<i>¿Qué es representación de datos?.....</i>	<i>27</i>
<i>Formas de representar datos</i>	<i>30</i>
<i>Qué es un outlier o valor atípico?.....</i>	<i>33</i>
<i>¿Qué es una consciencia del entorno?.....</i>	<i>38</i>
<i>¿Para que sirve un algoritmo de apoyo a la toma de decisiones?.....</i>	<i>42</i>
<i>¿Qué es la inteligencia artificial?</i>	<i>44</i>
<i>¿En que consiste el proceso de toma de decisiones?</i>	<i>48</i>
<i>¿En que cosiste el preprocesamiento de datos?.....</i>	<i>51</i>
<i>Fuentes consultadas.....</i>	<i>55</i>

Tratamiento de datos mediante listas de comprensión

1. Introducción

Las **listas de comprensión** (list comprehensions) son una característica poderosa y concisa en lenguajes de programación como Python, diseñada para crear listas nuevas a partir de secuencias o iterables existentes, aplicando transformaciones o filtros en una sola línea de código. Este método es ampliamente utilizado en el tratamiento de datos porque permite procesar colecciones de manera eficiente, elegante y legible, reduciendo la necesidad de bucles explícitos.

En el contexto del tratamiento de datos, las listas de comprensión son ideales para tareas como:

- Filtrar datos según condiciones específicas.
- Transformar elementos (por ejemplo, aplicar una función a cada elemento).
- Crear subconjuntos de datos de forma dinámica.

Su sintaxis compacta no solo mejora la legibilidad del código, sino que también puede optimizar el rendimiento en comparación con métodos tradicionales como los bucles for.

2. Desarrollo

2.1. Sintaxis básica

La sintaxis de una lista de comprensión en Python sigue este formato:

```
[expresión for elemento in iterable if condición]
```

- **expresión**: La operación o transformación que se aplica a cada elemento.
- **elemento**: La variable que representa cada ítem del iterable.
- **iterable**: La colección de datos de entrada (lista, tupla, rango, etc.).
- **condición** (opcional): Un filtro que determina si el elemento se incluye en la lista resultante.

Ejemplo básico: Supongamos que queremos crear una lista con los cuadrados de los números del 1 al 5:

```
cuadrados = [x**2 for x in range(1, 6)]  
print(cuadrados) # Salida: [1, 4, 9, 16, 25]
```

En lugar de usar un bucle tradicional como este:

```
cuadrados = []  
for x in range(1, 6):  
    cuadrados.append(x**2)
```

La lista de comprensión logra lo mismo en una sola línea, siendo más concisa.

2.2. Usos en el tratamiento de datos

Las listas de comprensión son especialmente útiles en el procesamiento de datos, ya que permiten manipular grandes conjuntos de datos de manera eficiente. A continuación, detallo algunos casos de uso comunes:

a) Filtrado de datos

Se pueden usar listas de comprensión para seleccionar elementos que cumplan ciertas condiciones.

Ejemplo: Filtrar números pares de una lista.

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8]  
pares = [x for x in numeros if x % 2 == 0]  
print(pares) # Salida: [2, 4, 6, 8]
```

b) Transformación de datos

Las listas de comprensión permiten aplicar una función o transformación a cada elemento.

Ejemplo: Convertir una lista de nombres a mayúsculas.

```
nombres = ["ana", "juan", "maría"]  
mayusculas = [nombre.upper() for nombre in nombres]  
print(mayusculas) # Salida: ['ANA', 'JUAN', 'MARÍA']
```

c) Anidamiento

Las listas de comprensión pueden incluir bucles anidados para trabajar con estructuras de datos más complejas, como listas de listas.

Ejemplo: Aplanar una lista de listas.

```
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
aplanada = [num for fila in matriz for num in fila]
print(aplanada) # Salida: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

d) Combinación con funciones

Se pueden combinar con funciones integradas o definidas por el usuario para procesar datos más complejos.

Ejemplo: Extraer las iniciales de una lista de nombres completos.

```
nombres_completos = ["Ana García", "Juan Pérez", "María López"]
iniciales = [nombre.split()[0][0] + nombre.split()[1][0] for nombre in nombres_completos]
print(iniciales) # Salida: ['AG', 'JP', 'ML']
```

2.3. Ventajas y limitaciones

Ventajas:

- **Concisión:** Reducen el código necesario en comparación con bucles tradicionales.
- **Legibilidad:** Un código bien escrito con listas de comprensión es más fácil de entender para programadores familiarizados con la sintaxis.
- **Eficiencia:** En muchos casos, son más rápidas que los bucles for debido a la optimización interna de Python.
- **Funcionalidad expresiva:** Facilitan operaciones funcionales como mapear y filtrar datos.

Limitaciones:

- **Complejidad:** Listas de comprensión muy anidadas o con condiciones complejas pueden volverse difíciles de leer.
- **Depuración:** Identificar errores en listas de comprensión puede ser más complicado que en un bucle explícito.
- **Memoria:** Para iterables muy grandes, las listas de comprensión generan la lista completa en memoria, lo que puede ser ineficiente (en estos casos, generadores o itertools podrían ser mejores).

Modulo Matplotlib

1. Introducción

Matplotlib es una biblioteca de código abierto en Python diseñada para crear visualizaciones de datos estáticas, interactivas y animadas. Es ampliamente utilizada en ciencia de datos, ingeniería, investigación científica y educación debido a su flexibilidad y capacidad para generar gráficos de alta calidad. Matplotlib permite crear una amplia variedad de gráficos, desde histogramas y gráficos de dispersión hasta visualizaciones 3D y mapas de calor, con un alto grado de personalización.

Desarrollada inicialmente por John D. Hunter en 2003, Matplotlib está inspirada en MATLAB, ofreciendo una interfaz similar pero con la ventaja de ser gratuita y de código abierto. Su módulo principal, `matplotlib.pyplot`, es el más utilizado para crear gráficos de manera sencilla, mientras que otros submódulos permiten funcionalidades avanzadas.

En el contexto del tratamiento de datos, Matplotlib es esencial para:

- Explorar datos visualmente (análisis exploratorio de datos).
- Presentar resultados de forma clara y profesional.
- Generar visualizaciones para informes, artículos científicos o aplicaciones interactivas.

2. Desarrollo

2.1. Instalación y configuración

Para usar Matplotlib, primero debes instalarlo. Esto se hace típicamente con `pip`:

```
pip install matplotlib
```

Una vez instalado, puedes importar el módulo `pyplot` (que es la interfaz más común) en tu script:

```
import matplotlib.pyplot as plt
```

Matplotlib funciona bien con otras bibliotecas como **NumPy** (para datos numéricos) y **Pandas** (para datos tabulares), lo que lo hace ideal para el ecosistema de ciencia de datos en Python.

2.2. Componentes principales

Matplotlib organiza sus gráficos en una jerarquía de objetos:

- **Figure:** El contenedor principal que contiene uno o más gráficos (como un lienzo).
- **Axes:** Cada subgráfico dentro de la figura, donde se dibujan los datos (ejes, líneas, puntos, etc.).
- **Axis:** Los ejes individuales (x, y) que definen las escalas y etiquetas.

Ejemplo básico de un gráfico de líneas:

```
import matplotlib.pyplot as plt
import numpy as np

# Datos
x = np.linspace(start=0, stop=10, num=100)
y = np.sin(x)

# Crear gráfico
plt.figure(figsize=(8, 6)) # Tamaño de la figura
plt.plot(*args=x, y, label='sin(x)', color='blue') # Graficar línea
plt.title('Función Seno') # Título
plt.xlabel('x') # Etiqueta eje x
plt.ylabel('sin(x)') # Etiqueta eje y
plt.legend() # Mostrar leyenda
plt.grid(True) # Mostrar cuadrícula
plt.show() # Mostrar gráfico
```

Este código genera un gráfico de la función seno, con etiquetas, título y una cuadrícula.

2.3. Tipos de gráficos principales

Matplotlib soporta una amplia variedad de gráficos. A continuación, algunos de los más comunes con ejemplos:

a) Gráfico de líneas (plt.plot)

Ideal para mostrar tendencias o relaciones continuas.

Ejemplo: Comparar dos funciones.

```
plt.plot(*args: x, y1, label='sin(x)', color='blue')
plt.plot(*args: x, y2, label='cos(x)', color='red', linestyle='--')
plt.title('Seno vs Coseno')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```

b) Gráfico de dispersión (plt.scatter)

Útil para mostrar relaciones entre dos variables.

Ejemplo: Puntos aleatorios.

```
np.random.seed(42)
x = np.random.rand(50)
y = np.random.rand(50)

plt.scatter(x, y, color='green', alpha=0.5)
plt.title('Gráfico de Dispersión')
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```


c) Histograma (plt.hist)

Para visualizar la distribución de datos.

Ejemplo: Distribución de datos aleatorios.

```
datos = np.random.randn(1000) # Datos normales
plt.hist(datos, bins=30, color='purple', alpha=0.7)
plt.title('Histograma de Datos Normales')
plt.xlabel('Valor')
plt.ylabel('Frecuencia')
plt.show()
```

d) Gráfico de barras (plt.bar)

Para comparar categorías.

Ejemplo: Comparación de ventas.

```
categorias = ['A', 'B', 'C']
valores = [10, 20, 15]

plt.bar(categorias, valores, color='orange')
plt.title('Ventas por Categoría')
plt.xlabel('Categoría')
plt.ylabel('Ventas')
plt.show()
```

e) Subgráficos (plt.subplots)

Para mostrar múltiples gráficos en una sola figura.

Ejemplo: Cuatro gráficos en una cuadrícula.

```
fig, axs = plt.subplots( nrows: 2, ncols: 2, figsize=(10, 8))
x = np.linspace( start: 0, stop: 10, num: 100)

axs[0, 0].plot(x, np.sin(x), 'b')
axs[0, 0].set_title('Seno')
axs[0, 1].plot(x, np.cos(x), 'r')
axs[0, 1].set_title('Coseno')
axs[1, 0].scatter(x, np.random.rand(100), color='g')
axs[1, 0].set_title('Dispersión')
axs[1, 1].hist(np.random.randn(100), bins=20, color='purple')
axs[1, 1].set_title('Histograma')

plt.tight_layout() # Ajustar espaciado
plt.show()
```

2.4. Personalización

Matplotlib ofrece un control extenso sobre la apariencia de los gráficos:

- **Colores y estilos:** Cambiar colores (color), estilos de línea (linestyle), marcadores, etc.
- **Escalas:** Escalas lineales, logarítmicas (plt.yscale('log')), etc.
- **Anotaciones:** Agregar texto, flechas o etiquetas (plt.annotate).
- **Temas:** Usar estilos predefinidos (plt.style.use('ggplot')) para cambiar la estética.

Ejemplo con personalización:

```
plt.tight_layout() # Ajustar espaciado
plt.show()

plt.style.use('seaborn') # Estilo similar a Seaborn
x = np.linspace( start: 0, stop: 10, num: 100)
y = np.sin(x)

plt.plot( *args: x, y, 'b-', label='sin(x)', linewidth=2)
plt.title( label: 'Gráfico Personalizado', fontsize=14, fontweight='bold')
plt.xlabel( xlabel: 'x', fontsize=12)
plt.ylabel( ylabel: 'sin(x)', fontsize=12)
plt.grid( visible: True, linestyle='--', alpha=0.7)
plt.legend(loc='upper right')
plt.show()
```

2.5. Ventajas y limitaciones

Ventajas:

- **Flexibilidad:** Permite personalizar casi cualquier aspecto de un gráfico.
- **Integración:** Funciona bien con NumPy, Pandas, y otras bibliotecas de ciencia de datos.
- **Amplia documentación:** Gran cantidad de tutoriales y ejemplos disponibles.
- **Soporte para exportación:** Gráficos exportables a formatos como PNG, PDF, SVG, etc.

Limitaciones:

- **Curva de aprendizaje:** La API puede ser compleja para usuarios nuevos, especialmente para gráficos avanzados.
- **Interactividad limitada:** Aunque soporta gráficos interactivos, otras bibliotecas como Plotly o Bokeh son más adecuadas para aplicaciones web.
- **Estética predeterminada:** Los gráficos base pueden parecer anticuados sin personalización.

2.6. Comparación con otras bibliotecas

- **Seaborn:** Construido sobre Matplotlib, ofrece gráficos más estéticos y simplificados para análisis estadístico.
- **Plotly:** Mejor para gráficos interactivos y aplicaciones web.
- **Bokeh:** Enfocado en visualizaciones interactivas para navegadores.
- **ggplot (Python):** Inspirado en R, usa una gramática de gráficos más declarativa.

Modulo Numpy

1. Introducción

NumPy (Numerical Python) es una biblioteca de código abierto en Python diseñada para realizar cálculos numéricos eficientes con arreglos multidimensionales (arrays) y matrices. Es la base de muchas otras bibliotecas de ciencia de datos, como Pandas, SciPy, y Matplotlib, debido a su capacidad para manejar grandes volúmenes de datos de manera rápida y versátil. NumPy ofrece funciones matemáticas avanzadas, herramientas de álgebra lineal, transformadas de Fourier, y generadores de números aleatorios, entre otras.

Creada por Travis Oliphant en 2005 (basada en bibliotecas previas como Numeric y Numarray), NumPy está optimizada para el rendimiento, utilizando código compilado en C bajo el capó. Es esencial en el tratamiento de datos porque permite:

- Manipular arreglos multidimensionales de forma eficiente.
- Realizar operaciones matemáticas vectorizadas, evitando bucles lentos en Python.
- Preparar datos para visualización (con Matplotlib) o modelado (con bibliotecas de machine learning).

2. Desarrollo

2.1. Instalación y configuración

Para usar NumPy, primero debes instalarlo (generalmente con pip):

```
pip install numpy
```

Una vez instalado, se importa con la convención estándar:

```
import numpy as np
```

2.2. Componente principal: El arreglo NumPy (ndarray)

El corazón de NumPy es el objeto ndarray (arreglo n-dimensional), que es una estructura de datos homogénea (todos los elementos tienen el mismo tipo) y multidimensional. A diferencia de las listas de Python, los arreglos NumPy son más eficientes para cálculos numéricos debido a:

- **Memoria contigua:** Los datos se almacenan de forma compacta, reduciendo el uso de memoria.
- **Vectorización:** Operaciones elemento por elemento sin bucles explícitos.
- **Tipos de datos definidos:** Soporta tipos como int32, float64, etc., optimizando cálculos.

Ejemplo básico: Crear un arreglo.

```
import numpy as np

# Crear un arreglo 1D
arr = np.array([1, 2, 3, 4])
print(arr) # Salida: [1 2 3 4]
print(arr.dtype) # Salida: int64
print(arr.shape) # Salida: (4,)
```



```
# Crear un arreglo 2D
matriz = np.array([[1, 2], [3, 4]])
print(matriz) # Salida: [[1 2]
                #          [3 4]]
print(matriz.shape) # Salida: (2, 2)
```

2.3. Creación de arreglos

NumPy ofrece varias formas de crear arreglos:

- **Desde listas o tuplas:**

```
lista = [1, 2, 3]
arr = np.array(lista)
```

Arreglos predefinidos:

```
zeros = np.zeros((2, 3)) # Matriz 2x3 de ceros
ones = np.ones((2, 3)) # Matriz 2x3 de unos
identidad = np.eye(3) # Matriz identidad 3x3
print(zeros) # Salida: [[0. 0. 0.]
# [0. 0. 0.]
```

Secuencias:

```
rango = np.arange(0, 10, 2) # Similar a range(0, 10, 2)
linspace = np.linspace(start: 0, stop: 1, num: 5) # 5 valores equiespaciados de 0 a 1
print(rango) # Salida: [0 2 4 6 8]
print(linspace) # Salida: [0. 0.25 0.5 0.75 1. ]
```

Aleatorios:

```
aleatorio = np.random.rand(2, 3) # Matriz 2x3 con valores entre 0 y 1
normal = np.random.randn(2, 3) # Matriz 2x3 con distribución normal
print(aleatorio) # Ejemplo: [[0.374 0.950 0.731]
# [0.598 0.156 0.155]]
```

2.4. Operaciones con arreglos

NumPy permite realizar operaciones matemáticas de forma vectorizada, lo que es mucho más rápido que usar bucles.

a) Operaciones elemento por elemento

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b) # Salida: [5 7 9]
print(a * b) # Salida: [4 10 18]
print(a ** 2) # Salida: [1 4 9]
```

b) Operaciones matriciales

```
matriz = np.array([[1, 2], [3, 4]])  
print(np.dot(matriz, matriz)) # Producto matricial  
# Salida: [[ 7 10]  
#          [15 22]]
```

c) Funciones universales (ufuncs)

NumPy ofrece funciones matemáticas que operan elemento por elemento:

```
arr = np.array([0, np.pi/2, np.pi])  
print(np.sin(arr)) # Salida: [0. 1. 0.]  
print(np.exp(arr)) # Salida: [ 1.      4.810 23.141]
```

d) Indexación y segmentación

Similar a las listas, pero con soporte multidimensional:

```
arr = np.array([[1, 2, 3], [4, 5, 6]])  
print(arr[0, 1])      # Salida: 2  
print(arr[:, 1])      # Salida: [2 5]  
print(arr[1, :2])     # Salida: [4 5]
```

e) Filtrado con condiciones

Usando máscaras booleanas:

```
arr = np.array([1, 2, 3, 4, 5])  
mascara = arr > 3  
print(arr[mascara]) # Salida: [4 5]
```

2.5. Integración con listas de comprensión

Aunque NumPy es más eficiente con operaciones vectorizadas, las listas de comprensión (como viste en tu primer tema) pueden combinarse con NumPy para tareas específicas:

Ejemplo: Crear un arreglo con una lista de comprensión.

```
datos = [x**2 for x in range(5)] # Lista de comprensión
arr = np.array(datos)
print(arr) # Salida: [ 0  1  4  9 16]
```

2.6. Ventajas y limitaciones

Ventajas:

- **Rendimiento:** Operaciones vectorizadas y uso de código C hacen que sea mucho más rápido que las listas de Python.
- **Flexibilidad:** Soporta arreglos multidimensionales y una amplia gama de operaciones matemáticas.
- **Interoperabilidad:** Base para Pandas, Matplotlib, TensorFlow, y otras bibliotecas.
- **Documentación extensa:** Gran cantidad de recursos y ejemplos.

Limitaciones:

- **Curva de aprendizaje:** La sintaxis puede ser intimidante para principiantes, especialmente con arreglos multidimensionales.
- **Memoria estática:** Los arreglos tienen un tamaño fijo, lo que puede requerir redimensionamiento explícito (`np.resize` o concatenación).
- **Funcionalidad limitada para datos heterogéneos:** Para datos tabulares con tipos mixtos, Pandas es más adecuado.

2.7. Comparación con otras herramientas

- **Listas de Python:** Más lentas y menos eficientes para cálculos numéricos, pero más flexibles para datos heterogéneos.
- **Pandas:** Construido sobre NumPy, es mejor para datos tabulares y análisis estadístico.
- **SciPy:** Extiende NumPy con funciones avanzadas para optimización, integración, y estadística.
- **TensorFlow/PyTorch:** Optimizados para machine learning, usan estructuras similares a ndarray pero con soporte para GPU.

Instrucción eval en python

1. Introducción

La función **eval** en Python es una herramienta incorporada que permite **evaluar expresiones dinámicas** escritas como cadenas de texto (strings) y ejecutarlas como si fueran código Python válido. Es útil para procesar expresiones matemáticas o fragmentos de código generados en tiempo de ejecución, pero su uso debe ser cuidadosamente controlado debido a **riesgos de seguridad** significativos, especialmente cuando se manejan entradas de usuarios. En el contexto del tratamiento de datos, eval puede ser utilizado para:

- Evaluar expresiones matemáticas introducidas por el usuario (por ejemplo, "2 + 3 * 4").
- Procesar configuraciones dinámicas o fórmulas almacenadas como texto.
- Ejecutar código generado programáticamente (aunque esto es raro y generalmente desaconsejado).

Sin embargo, debido a sus implicaciones de seguridad, eval suele evitarse en favor de alternativas más seguras como `ast.literal_eval` o bibliotecas específicas para parsing.

2. Desarrollo

2.1. Sintaxis y funcionamiento

La función eval tiene la siguiente sintaxis:

```
eval(expression, globals=None, locals=None)
```

- **expression**: Una cadena de texto que contiene una expresión Python válida (por ejemplo, '2 + 3').
- **globals** (opcional): Un diccionario que define el entorno global de variables disponible para la evaluación.
- **locals** (opcional): Un diccionario que define el entorno local de variables.
- Devuelve el resultado de evaluar la expresión.

Ejemplo básico:

```
resultado = eval('2 + 3')  
print(resultado) # Salida: 5
```

En este caso, eval interpreta la cadena '2 + 3' como una expresión matemática y devuelve el resultado.

2.2. Usos comunes

a) Evaluación de expresiones matemáticas

eval es útil para calcular expresiones matemáticas dinámicas.

Ejemplo:

```
expresion = input("Ingresa una expresión matemática: ") # Ejemplo: "5 * (3 + 2)"
resultado = eval(expresion)
print(resultado) # Salida: 25
```

b) Acceso a variables dinámicas

eval puede usarse para acceder a variables o ejecutar código basado en nombres generados dinámicamente.

Ejemplo:

```
x = 10
y = 20
variable = 'x + y'
resultado = eval(variable)
print(resultado) # Salida: 30
```

2.3. Ventajas y limitaciones

Ventajas:

- **Flexibilidad:** Permite ejecutar código dinámico sin necesidad de escribirlo explícitamente.
- **Simplicidad:** Útil para prototipos rápidos o evaluación de expresiones matemáticas simples.
- **Integración:** Compatible con el ecosistema de Python (NumPy, etc.).

Limitaciones:

- **Seguridad:** Extremadamente peligroso con entradas no confiables.
- **Rendimiento:** Puede ser más lento que soluciones específicas (como numexpr para matemáticas).
- **Depuración:** Los errores en las expresiones evaluadas pueden ser difíciles de rastrear.

¿Qué es Adquisición de Datos?

1. Introducción

La **Adquisición de Datos** (Data Acquisition, DAQ) se refiere al proceso de **recopilar, medir y registrar datos** del mundo real o de sistemas digitales para su posterior análisis, visualización o procesamiento. Este proceso es fundamental en ciencia de datos, ingeniería, investigación científica e industria, ya que proporciona los datos crudos necesarios para tomar decisiones informadas, modelar fenómenos o monitorear sistemas.

En términos generales, la adquisición de datos implica:

- **Medir señales** (físicas o digitales) utilizando sensores, dispositivos o fuentes de datos.
- **Convertir estas señales** en un formato digital comprensible (si son analógicas).
- **Almacenar o procesar** los datos para su uso en análisis o visualización.

En el contexto del tratamiento de datos con Python, la adquisición de datos es el primer paso en el flujo de trabajo, seguido por el procesamiento (por ejemplo, con NumPy o listas de comprensión) y la visualización (con Matplotlib).

2. Desarrollo

2.1. Componentes de la Adquisición de Datos

Un sistema de adquisición de datos típicamente incluye los siguientes elementos:

1. Fuentes de datos:

- **Señales físicas:** Temperatura, presión, voltaje, luz, sonido, etc., medidas por sensores.
- **Señales digitales:** Datos de APIs, bases de datos, archivos (CSV, JSON, etc.), o streams en tiempo real (por ejemplo, redes sociales).
- **Datos generados:** Simulaciones o datos sintéticos creados mediante algoritmos.

2. Sensores o dispositivos de medición:

- Para señales físicas, se utilizan sensores como termopares, acelerómetros o fotodiodos.
- Para datos digitales, se emplean interfaces como APIs REST, web scraping o conexiones a bases de datos.

3. Conversión de señales:

- **Análogo a Digital (ADC):** Convierte señales analógicas (continuas) en datos digitales (discretos) mediante muestreo.
- **Preprocesamiento:** Incluye filtrado de ruido, amplificación o normalización.

4. Hardware de adquisición:

- Dispositivos DAQ (como los de National Instruments) que conectan sensores a computadoras.
- Interfaces como Arduino, Raspberry Pi o sistemas embebidos para proyectos más pequeños.

5. Software:

- Programas para controlar la adquisición, almacenar datos y procesarlos. En Python, bibliotecas como pandas, numpy o pydaq son comunes.
- Interfaz con hardware mediante bibliotecas como pyvisa o pyserial.

6. Almacenamiento:

- Los datos se guardan en archivos (CSV, HDF5), bases de datos (SQL, NoSQL) o en memoria para procesamiento en tiempo real.

2.2. Métodos de Adquisición de Datos

La adquisición de datos puede realizarse de varias formas, dependiendo de la fuente y el propósito:

1. Adquisición en tiempo real:

- Datos capturados continuamente desde sensores o streams (por ejemplo, monitoreo de temperatura en una fábrica).
- Ejemplo: Usar un Arduino para leer un sensor de temperatura cada segundo.

2. Adquisición por lotes:

- Recopilación de datos en un momento específico o desde fuentes estáticas (por ejemplo, descargar un archivo CSV).
- Ejemplo: Importar un conjunto de datos desde Kaggle.

3. Adquisición automatizada:

- Uso de scripts o sistemas para recolectar datos automáticamente (por ejemplo, web scraping o consultas a APIs).
- Ejemplo: Extraer publicaciones de X con una API.

4. Adquisición manual:

- Entrada de datos por usuarios, como formularios o digitación de mediciones.
- Ejemplo: Ingresar datos experimentales en una hoja de cálculo.

2.3. Ejemplos prácticos en Python

Dado tu interés en Python, aquí hay ejemplos de adquisición de datos usando herramientas comunes:

a) Desde un archivo CSV

Los archivos CSV son una fuente común de datos.

```
import pandas as pd

# Leer datos desde un CSV
datos = pd.read_csv('datos.csv')
print(datos.head()) # Mostrar las primeras filas
```

b) Desde una API

Adquirir datos en tiempo real desde una API pública (por ejemplo, una API de clima).

```
import requests

# Obtener datos de una API de clima (ejemplo)
url = "https://api.openweathermap.org/data/2.5/weather?q=London&appid=tu_api_key"
respuesta = requests.get(url)
datos = respuesta.json()
print(datos['main']['temp']) # Temperatura en Londres
```

Nota: Necesitas una clave de API para este ejemplo.

c) Desde un sensor con Arduino

Usando pyserial para leer datos de un sensor conectado a un Arduino.

```
import serial

# Conectar al puerto del Arduino
ser = serial.Serial(port='COM3', baudrate=9600) # Ajusta el puerto según tu sistema

# Leer datos
datos = []
for _ in range(10):
    linea = ser.readline().decode().strip() # Leer línea del sensor
    datos.append(float(linea)) # Convertir a número

print(datos) # Lista de mediciones
ser.close()
```

Nota: Requiere un Arduino configurado para enviar datos por el puerto serial.

d) Integración con NumPy y Matplotlib

Adquirir datos y visualizarlos.

```
import numpy as np
import matplotlib.pyplot as plt

# Simular datos de un sensor (adquisición sintética)
t = np.linspace( start: 0, stop: 10, num: 100)
datos = np.sin(t) + np.random.normal( loc: 0, scale: 0.1, size: 100) # Seno con ruido

# Visualizar
plt.plot( *args: t, datos, label='Datos adquiridos')
plt.title('Adquisición de Datos Simulada')
plt.xlabel('Tiempo (s)')
plt.ylabel('Valor')
plt.legend()
plt.grid(True)
plt.show()
```

e) Con listas de comprensión

Filtrar datos adquiridos usando listas de comprensión.

```
# Supongamos que adquirimos datos desde un archivo
datos = [1.2, 3.5, -0.1, 4.7, 2.3, -1.0]

# Filtrar valores positivos con lista de comprensión
positivos = [x for x in datos if x > 0]
print(positivos) # Salida: [1.2, 3.5, 4.7, 2.3]
```

2.4. Ventajas y limitaciones

Ventajas:

- **Base para análisis:** Proporciona los datos crudos necesarios para cualquier proyecto de ciencia de datos o ingeniería.
- **Versatilidad:** Aplicable a una amplia gama de fuentes, desde sensores físicos hasta datos digitales.
- **Automatización:** Puede integrarse con sistemas modernos para recopilar datos en tiempo real.

Limitaciones:

- **Calidad de los datos:** Los datos adquiridos pueden contener ruido, valores faltantes o errores que requieren limpieza.
- **Costo:** Los sistemas DAQ de alta precisión (hardware) pueden ser costosos.
- **Complejidad técnica:** Configurar sensores o interfaces requiere conocimientos específicos.

¿Qué es entorno?

1. Introducción

En programación, un **entorno** se refiere al **contexto en el que se ejecuta un programa**, incluyendo las variables, bibliotecas, configuraciones y recursos disponibles durante su ejecución. En Python, el término "entorno" suele asociarse con dos conceptos principales:

1. **Entorno de ejecución:** El conjunto de variables, funciones y objetos disponibles en un momento dado durante la ejecución de un programa, definido por los ámbitos (scopes) global y local.
2. **Entorno virtual:** Un espacio aislado que contiene una instalación específica de Python y sus bibliotecas, usado para gestionar dependencias de proyectos.

Ambos conceptos son cruciales en el tratamiento de datos, ya que determinan cómo se ejecutan herramientas como NumPy, Matplotlib, o scripts que usan eval, y cómo se organizan las dependencias para proyectos de adquisición de datos.

En el contexto de tus consultas previas:

- **Entorno de ejecución** es relevante para eval, ya que controla las variables accesibles (globals y locals).
- **Entornos virtuales** son esenciales para gestionar bibliotecas como NumPy y Matplotlib, asegurando que las versiones correctas estén disponibles sin conflictos.
- En **adquisición de datos**, el entorno puede incluir el hardware y software que interactúan con los sensores o fuentes de datos.

2. Desarrollo

2.1. Entorno de ejecución

El **entorno de ejecución** en Python se refiere al contexto donde se evalúan las expresiones y se ejecutan las instrucciones. Está definido por:

- **Ámbitos (scopes):** Regiones del código donde las variables y funciones son accesibles.
 - **Global:** Variables definidas a nivel de módulo.
 - **Local:** Variables definidas dentro de una función.
 - **Enclosing:** Variables en funciones anidadas.
 - **Built-in:** Funciones y objetos predefinidos de Python (como print o len).
- **Namespaces:** Diccionarios que mapean nombres a objetos (por ejemplo, globals() y locals()).

Ejemplo básico:

```
x = 10 # Variable global

def mi_funcion(): 1 usage
    y = 20 # Variable local
    print(f"Global: {globals()['x']}")
    print(f"Local: {locals()['y']}")

mi_funcion()
# Salida:
# Global: 10
# Local: 20
```

2.2. Entornos virtuales

Un **entorno virtual** es un directorio aislado que contiene una copia de Python y un conjunto de bibliotecas específicas para un proyecto. Esto evita conflictos entre dependencias (por ejemplo, si un proyecto requiere NumPy 1.21 y otro NumPy 1.23).

Creación de un entorno virtual

Python proporciona el módulo venv para crear entornos virtuales:

```
# Crear un entorno virtual
python -m venv mi_entorno

# Activar el entorno (Windows)
mi_entorno\Scripts\activate

# Activar el entorno (Linux/Mac)
source mi_entorno/bin/activate
```

Una vez activado, el entorno virtual usa su propia instalación de pip y Python.

Ejemplo práctico: Supongamos que quieres usar NumPy y Matplotlib en un proyecto.

```
# Crear y activar entorno
python -m venv proyecto_datos
source proyecto_datos/bin/activate # Linux/Mac

# Instalar bibliotecas
pip install numpy matplotlib

# Verificar instalación
pip list
```

Ahora, puedes usar NumPy y Matplotlib sin afectar otros proyectos.

Relación con temas previos

- **NumPy y Matplotlib:** Los entornos virtuales aseguran que las versiones correctas de estas bibliotecas estén disponibles. Por ejemplo, un proyecto de visualización puede requerir una versión específica de Matplotlib compatible con tu código.
- **Adquisición de datos:** Los entornos virtuales son útiles para instalar bibliotecas como pyserial (para sensores) o requests (para APIs) sin conflictos.
- **Listas de comprensión:** Aunque no dependen directamente de entornos virtuales, el código que las usa a menudo se ejecuta en un entorno con bibliotecas específicas instaladas.

Ejemplo integrando temas:

```
# En un entorno virtual con NumPy y Matplotlib instalados
import numpy as np
import matplotlib.pyplot as plt

# Adquisición de datos simulada
t = np.linspace(start=0, stop=10, num=100)
datos = [np.sin(x) + np.random.normal(loc=0, scale=0.1) for x in t] # Lista de comprensión

# Visualización
plt.plot(*args=t, datos, label='Datos adquiridos')
plt.title('Adquisición y Visualización en Entorno Virtual')
plt.xlabel('Tiempo (s)')
plt.ylabel('Valor')
plt.legend()
plt.grid(True)
plt.show()
```

2.3. Otros significados de "entorno"

Dependiendo del contexto, "entorno" puede referirse a:

1. Entorno de desarrollo:

- Herramientas como Jupyter Notebook, VS Code, o PyCharm configuradas para trabajar con Python.
- Ejemplo: Configurar un entorno en Jupyter para usar NumPy y Matplotlib.

2. Entorno en adquisición de datos:

- El sistema físico o digital donde se recopilan datos (por ejemplo, sensores en un laboratorio o una API en la nube).
- Ejemplo: Un Arduino conectado a un sensor de temperatura forma parte del entorno de adquisición.

3. Entorno de datos:

- El ecosistema de herramientas y plataformas para gestionar datos (por ejemplo, bases de datos, pipelines de ETL).
- Ejemplo: Un entorno que combina Python, SQL y Apache Spark para procesar grandes volúmenes de datos.

2.4. Herramientas para gestionar entornos

Además de venv, existen otras herramientas para manejar entornos virtuales:

- **virtualenv**: Similar a venv, pero más configurable.
- **Conda**: Gestiona entornos y dependencias, especialmente útil para ciencia de datos (soporta NumPy, Matplotlib, etc.).

2.5. Ventajas y limitaciones

Ventajas:

- **Aislamiento**: Evita conflictos entre proyectos con diferentes versiones de bibliotecas.
- **Reproducibilidad**: Facilita compartir proyectos con dependencias exactas.
- **Flexibilidad**: Permite experimentar con nuevas bibliotecas sin afectar el sistema global.
- **Seguridad**: Limita el acceso de eval a variables controladas en entornos de ejecución.

Limitaciones:

- **Espacio en disco**: Cada entorno virtual ocupa espacio adicional.
- **Configuración inicial**: Puede ser intimidante para principiantes.
- **Gestión compleja**: Proyectos con muchas dependencias requieren herramientas avanzadas como Conda o Poetry.

¿Qué es representación de datos?

1. Introducción

La **representación de datos** se refiere al proceso de **organizar, estructurar y presentar datos** de manera que puedan ser almacenados, procesados, analizados y comunicados eficazmente. En informática y ciencia de datos, esto implica:

- **Estructuras de datos:** Cómo se organizan los datos en memoria (por ejemplo, listas, arreglos, diccionarios).
- **Formatos de almacenamiento:** Cómo se guardan los datos en disco o bases de datos (por ejemplo, CSV, JSON, HDF5).
- **Visualización:** Cómo se presentan los datos para que sean comprensibles (por ejemplo, gráficos con Matplotlib).
- **Codificación interna:** Cómo se representan los datos a nivel binario (por ejemplo, enteros, flotantes).

En el contexto del tratamiento de datos, la representación es crucial porque determina cómo se manipulan los datos (con herramientas como NumPy o listas de comprensión) y cómo se comunican los resultados (con Matplotlib). Una buena representación facilita el análisis, reduce errores y optimiza el rendimiento.

2. Desarrollo

2.1. Componentes de la Representación de Datos

1. **Estructuras de datos:** Las estructuras de datos son formas de organizar datos en memoria para su procesamiento eficiente. En Python, incluyen:
 - **Listas:** Colecciones ordenadas y mutables, útiles para datos secuenciales.

```
lista = [1, 2, 3, 4]
```

2.Formatos de almacenamiento: Los datos se representan en disco o bases de datos en formatos específicos:

- **CSV:** Tablas separadas por comas, ideales para datos tabulares.
- **JSON:** Formato ligero para datos estructurados, común en APIs.
- **HDF5:** Formato eficiente para grandes volúmenes de datos, usado con NumPy y Pandas.
- **Bases de datos:** SQL (relacionales) o NoSQL (como MongoDB) para datos estructurados o semi-estructurados.

3.Codificación interna: A nivel bajo, los datos se representan en bits:

- **Números:** Enteros (int32, int64), flotantes (float64).
- **Texto:** Codificaciones como UTF-8 o ASCII.
- **Datos complejos:** Estructuras binarias para imágenes, audio, etc.

3. Visualización: La representación visual transforma datos en gráficos o diagramas para facilitar su interpretación. En Python, Matplotlib es una herramienta clave para esto.

2.2. Representación en Python

a) Estructuras de datos con listas de comprensión

Las listas de comprensión (como viste previamente) son una forma de representar datos transformados.

Ejemplo: Convertir datos crudos en una nueva representación.

```
# Datos crudos
temperaturas = [20, 22, 19, 25]

# Representar como temperaturas normalizadas
norm_temps = [(t - min(temperaturas)) / (max(temperaturas) - min(temperaturas)) for t in temperaturas]
print(norm_temps) # Salida: [0.16666666666666666, 0.5, 0.0, 1.0]
```

b) Relación con entornos

El entorno (virtual o de ejecución) determina qué herramientas están disponibles para representar datos. Por ejemplo, un entorno virtual con NumPy y Matplotlib instalados permite representar datos como arreglos y gráficos.

2.3. Tipos de Representación

1. **Númerica:**
 - Datos representados como números (enteros, flotantes) en arreglos o matrices.
 - Ejemplo: Arreglos NumPy para cálculos científicos.
2. **Tabular:**
 - Datos organizados en filas y columnas (como en hojas de cálculo).
 - Ejemplo: DataFrames de Pandas.
3. **Gráfica:**
 - Datos representados como gráficos, histogramas, o mapas de calor.
 - Ejemplo: Gráficos de Matplotlib.
4. **Texto:**
 - Datos representados como cadenas o documentos.
 - Ejemplo: JSON o logs de sensores.
5. **Estructurada:**
 - Datos organizados en objetos complejos (como árboles o grafos).
 - Ejemplo: Diccionarios anidados para datos jerárquicos.

2.4. Ventajas y limitaciones

Ventajas:

- **Claridad:** Una buena representación hace que los datos sean comprensibles.
- **Eficiencia:** Estructuras como arreglos NumPy optimizan el procesamiento.
- **Comunicación:** La visualización facilita compartir hallazgos.
- **Flexibilidad:** Múltiples formatos y estructuras para diferentes necesidades.

Limitaciones:

- **Complejidad:** Elegir la representación adecuada requiere conocimiento del problema.
- **Pérdida de información:** Algunas representaciones (como la agregación) pueden simplificar datos y omitir detalles.
- **Recursos:** Representaciones complejas (como visualizaciones 3D) consumen más memoria y tiempo.

Formas de representar datos

1. Introducción

La **representación de datos** se refiere al proceso de organizar, estructurar y presentar datos para que sean comprensibles, procesables y comunicables. En el contexto del tratamiento de datos, las formas de representar datos son esenciales para transformar los datos crudos obtenidos en la **adquisición de datos** en formatos que permitan análisis, almacenamiento o comunicación. Estas formas incluyen:

- **Estructuras de datos:** Cómo se organizan los datos en memoria (por ejemplo, listas, diccionarios, conjuntos).
- **Formatos de almacenamiento:** Cómo se guardan los datos en disco o bases de datos (por ejemplo, CSV, JSON).
- **Codificación interna:** Cómo se representan los datos a nivel binario (por ejemplo, enteros, cadenas).
- **Visualización:** Cómo se presentan los datos gráficamente (usaré ejemplos sin Matplotlib, como texto o herramientas alternativas).

En relación con tus temas previos:

- **Listas de comprensión:** Transforman datos en nuevas representaciones dentro de estructuras como listas.
- **Adquisición de datos:** Proporciona los datos crudos que se representan en estructuras o formatos.
- **Entornos:** Determinan las herramientas disponibles para la representación (por ejemplo, Python estándar o Pandas en un entorno virtual).

A continuación, detallo las principales formas de representar datos, con ejemplos en Python que evitan Matplotlib, NumPy y eval.

2. Desarrollo

2.1. Estructuras de Datos

Las estructuras de datos son formas de organizar datos en memoria para su procesamiento eficiente. En Python, las estructuras básicas incluyen:

1. Listas:

- Colecciones ordenadas y mutables, ideales para datos secuenciales.
- **Ejemplo:**

```
temperaturas = [20, 22, 19, 25]
print(temperaturas) # Salida: [20, 22, 19, 25]
```

Con listas de comprensión:

```
# Representar temperaturas normalizadas
min_temp = min(temperaturas)
max_temp = max(temperaturas)
norm_temps = [(t - min_temp) / (max_temp - min_temp) for t in temperaturas]
print(norm_temps) # Salida: [0.16666666666666666, 0.5, 0.0, 1.0]
```

2.2. Formatos de Almacenamiento

Los datos se representan en disco o bases de datos en formatos específicos para su almacenamiento y recuperación:

1. CSV (Comma-Separated Values):

- Formato tabular simple, ampliamente usado para datos estructurados.
- **Ejemplo:**

```
import csv

# Representar datos en un archivo CSV
datos = [['Nombre', 'Edad'], ['Ana', 25], ['Juan', 30]]
✓ with open('datos.csv', 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerows(datos)

# Leer y representar datos
✓ with open('datos.csv', 'r') as file:
    reader = csv.reader(file)
    datos_leidos = [row for row in reader] # Lista de comprensión
print(datos_leidos)
# Salida: [['Nombre', 'Edad'], ['Ana', '25'], ['Juan', '30']]
```


JSON (JavaScript Object Notation):

- Formato ligero para datos estructurados, común en APIs y configuraciones.
- **Ejemplo:**

```
import json

# Representar datos como JSON
datos = {'estudiantes': [{'nombre': 'Ana', 'edad': 25}, {'nombre': 'Juan', 'edad': 30}]}
with open('datos.json', 'w') as file:
    json.dump(datos, file)

# Leer y representar datos
with open('datos.json', 'r') as file:
    datos_leidos = json.load(file)
print(datos_leidos)

# Salida: {'estudiantes': [{'nombre': 'Ana', 'edad': 25}, {'nombre': 'Juan', 'edad': 30}]}
```

2.3. Ventajas y Limitaciones

Ventajas:

- **Flexibilidad:** Múltiples estructuras y formatos para diferentes necesidades.
- **Claridad:** Representaciones como tablas o texto facilitan la interpretación.
- **Eficiencia:** Estructuras como listas y diccionarios son rápidas para pequeños datasets.
- **Comunicación:** Representaciones textuales son universales y no requieren herramientas gráficas.

Limitaciones:

- **Escalabilidad:** Las listas y diccionarios pueden ser ineficientes para grandes volúmenes de datos (aquí NumPy sería útil, pero lo evitamos).
- **Visualización limitada:** Sin Matplotlib, las representaciones gráficas son rudimentarias.
- **Complejidad:** Elegir la representación adecuada requiere entender el problema.

2.4. Comparación con Otras Etapas

- **Adquisición vs. Representación:** La adquisición recopila datos crudos; la representación los organiza en estructuras o formatos.
- **Representación vs. Procesamiento:** La representación estructura los datos; el procesamiento los transforma (por ejemplo, con listas de comprensión).
- **Representación vs. Almacenamiento:** La representación en memoria (listas) precede al almacenamiento en disco (CSV, JSON).

Qué es un outlier o valor atípico?

1. Introducción

Un **outlier** o **valor atípico** es un dato que se **desvía significativamente** del resto de los valores en un conjunto de datos. Estos valores son inusuales en comparación con la distribución general de los datos y pueden indicar errores, anomalías o eventos raros. En el contexto del tratamiento de datos, identificar y manejar outliers es crucial para garantizar la calidad del análisis, ya que pueden distorsionar estadísticas, modelos o visualizaciones.

En relación con tus temas previos:

- **Adquisición de datos:** Los outliers pueden surgir de errores en la medición (por ejemplo, un sensor defectuoso) o eventos reales (como un pico de temperatura).
- **Representación de datos:** Los outliers afectan cómo se estructuran o visualizan los datos (por ejemplo, en tablas o estadísticas).
- **Listas de comprensión:** Pueden usarse para filtrar o detectar outliers en un conjunto de datos.
- **Entornos:** Las herramientas para identificar outliers (como Pandas) dependen del entorno virtual configurado.

2. Desarrollo

2.1. Definición y Características

Un **outlier** es un valor que se encuentra fuera del rango esperado de un conjunto de datos, según criterios estadísticos o contextuales. Por ejemplo:

- En un conjunto de temperaturas [20, 22, 19, 25, 100], el valor 100 es un outlier porque está muy lejos de los demás.
- En un conjunto de tiempos de respuesta [1.2, 1.3, 1.1, 10.5], el valor 10.5 es atípico.

Características:

- **Desviación significativa:** Los outliers están lejos de la media, mediana o rango intercuartílico.
- **Impacto:** Pueden sesgar estadísticas como la media o la varianza.
- **Origen:** Pueden ser errores (fallos en la adquisición), anomalías válidas (eventos raros) o ruido.

2.2. Causas de los Outliers

Los valores atípicos pueden originarse por:

1. **Errores de medición:** Fallos en sensores o entrada manual incorrecta.
 - Ejemplo: Un sensor de temperatura registra 1000°C debido a un cortocircuito.
2. **Errores de datos:** Errores tipográficos o problemas en la adquisición.
 - Ejemplo: Ingresar 100 en lugar de 10.
3. **Eventos raros:** Fenómenos legítimos pero inusuales.
 - Ejemplo: Un pico de tráfico en un sitio web debido a un evento viral.

4. **Distribuciones no uniformes:** En datasets con colas pesadas, algunos valores extremos son naturales.

- Ejemplo: Ingresos de una población, donde pocos tienen ingresos muy altos.

2.3. Métodos para Identificar Outliers

Aunque herramientas como NumPy facilitarían los cálculos estadísticos, usaré métodos simples basados en Python estándar o listas de comprensión para cumplir con tu restricción.

Los métodos comunes incluyen:

1. Regla del rango intercuartílico (IQR):

- Calcula el primer cuartil (Q1), el tercer cuartil (Q3) y el rango intercuartílico ($IQR = Q3 - Q1$).
- Un valor es outlier si está fuera de $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$.

Ejemplo:

```
# Datos
datos = [20, 22, 19, 25, 100, 21, 23]

# Ordenar datos
sorted_datos = sorted(datos)

# Calcular Q1, Q2 (mediana), Q3
n = len(sorted_datos)
Q1 = sorted_datos[n//4] # Aproximación simple
Q3 = sorted_datos[3*n//4]
IQR = Q3 - Q1

# Límites para outliers
limite_inferior = Q1 - 1.5 * IQR
limite_superior = Q3 + 1.5 * IQR

# Detectar outliers con lista de comprensión
outliers = [x for x in datos if x < limite_inferior or x > limite_superior]
print(outliers) # Salida: [100]
```

2. Desviación respecto a la media:

- Un valor es outlier si está a más de un número fijo de desviaciones estándar de la media (por ejemplo, 3).
- Sin NumPy, calculamos la media y desviación estándar manualmente.

Ejemplo:

```
datos = [20, 22, 19, 25, 100, 21, 23]

# Calcular media
media = sum(datos) / len(datos)

# Calcular desviación estándar (simplificada)
varianza = sum((x - media) ** 2 for x in datos) / len(datos)
desv_est = varianza ** 0.5

# Detectar outliers (±3 desviaciones estándar)
outliers = [x for x in datos if abs(x - media) > 3 * desv_est]
print(outliers) # Salida: [100]
```

☐ Inspección manual o contextual:

- En datasets pequeños, los outliers pueden identificarse visualmente o según conocimiento del dominio.
- Ejemplo: En [1, 2, 3, 999], 999 es claramente atípico.

☐ Con Pandas (opcional, para datasets grandes):

- Pandas simplifica el cálculo del IQR.
- Ejemplo:

```
import pandas as pd

datos = pd.Series([20, 22, 19, 25, 100, 21, 23])
Q1 = datos.quantile(0.25)
Q3 = datos.quantile(0.75)
IQR = Q3 - Q1
outliers = datos[(datos < Q1 - 1.5 * IQR) | (datos > Q3 + 1.5 * IQR)]
print(outliers.tolist()) # Salida: [100]
```

2.4. Manejo de Outliers

Una vez identificados, los outliers pueden manejarse de varias formas:

1. Eliminarlos:

- Adecuado si son errores.
- **Ejemplo** con lista de comprensión:

```
datos = [20, 22, 19, 25, 100, 21, 23]
datos_limpios = [x for x in datos if x != 100]
print(datos_limpios) # Salida: [20, 22, 19, 25, 21, 23]
```

2. Sustituirlos:

- Reemplazar con la media, mediana o un valor interpolado.
- **Ejemplo:**

```
datos = [20, 22, 19, 25, 100, 21, 23]
mediana = sorted(datos)[len(datos)//2]
datos_corregidos = [mediana if x == 100 else x for x in datos]
print(datos_corregidos) # Salida: [20, 22, 19, 25, 22, 21, 23]
```

3. Mantenerlos:

- Si son válidos (por ejemplo, un evento raro), se analizan por separado.
- Ejemplo: Un pico de tráfico web puede ser un outlier pero relevante.

4. Transformación:

- Aplicar transformaciones (como logaritmos) para reducir el impacto de outliers (sin NumPy, esto es más manual).

2.5. Ventajas y Limitaciones

Ventajas:

- **Mejor calidad de datos:** Eliminar o corregir outliers mejora la precisión de los análisis.
- **Detección de anomalías:** Los outliers pueden revelar eventos importantes (por ejemplo, fraudes).
- **Simplicidad:** Métodos como el IQR son fáciles de implementar con Python básico.

Limitaciones:

- **Subjetividad:** Definir un outlier depende del contexto y el método.
- **Pérdida de información:** Eliminar outliers válidos puede sesgar los resultados.
- **Cálculos manuales:** Sin NumPy, los métodos estadísticos son más laboriosos.

2.6. Comparación con Otras Etapas

- **Adquisición vs. Outliers:** Los outliers suelen detectarse tras la adquisición, como parte de la limpieza de datos.
- **Representación vs. Outliers:** Los outliers pueden distorsionar representaciones (por ejemplo, estadísticas en tablas).
- **Procesamiento vs. Outliers:** Filtrar outliers es un paso previo al procesamiento con listas de comprensión u otras herramientas.

¿Qué es una consciencia del entorno?

1. Introducción

En el contexto del tratamiento de datos y la programación, la **consciencia del entorno** se refiere a la capacidad de un programa, script o sistema para **percibir, interpretar y reaccionar** al entorno en el que opera. Este "entorno" puede incluir:

- **Entorno de ejecución:** Las variables, configuraciones y recursos disponibles durante la ejecución de un programa (por ejemplo, variables globales, sistema operativo, o hardware).
- **Datos adquiridos:** La información recopilada del mundo real o sistemas digitales (como en la adquisición de datos).
- **Contexto del sistema:** La configuración del entorno virtual, las dependencias instaladas, o las interacciones con otros sistemas.

En ciencia de datos, la consciencia del entorno implica que un programa pueda:

- Acceder a datos relevantes (por ejemplo, archivos CSV o APIs).
- Adaptarse a cambios en los datos o el sistema (por ejemplo, detectar outliers).
- Interactuar con el entorno de manera dinámica (por ejemplo, leer configuraciones del sistema operativo).

En relación con tus temas previos:

- **Adquisición de datos:** La consciencia del entorno incluye "percibir" los datos crudos provenientes de sensores, archivos o APIs.
- **Entornos:** Los entornos virtuales o de ejecución definen el contexto que el programa "entiende".
- **Listas de comprensión:** Pueden usarse para procesar datos del entorno de forma dinámica.
- **Representación de datos:** La consciencia del entorno permite representar datos de manera adecuada según el contexto.
- **Outliers:** Detectar valores atípicos requiere "comprender" la distribución de los datos en el entorno.

2. Desarrollo

2.1. Consciencia del Entorno en Programación

En programación, la consciencia del entorno se refiere a cómo un programa accede e interactúa con su contexto de ejecución. Esto incluye:

1. Acceso a variables y configuraciones:

- Un programa puede consultar variables globales, locales o configuraciones del sistema.
- **Ejemplo:**

```
import os

# "Percibir" el directorio de trabajo actual
directorio = os.getcwd()
print(directorio) # Salida: /ruta/actual

# Listar archivos en el entorno
archivos = [f for f in os.listdir()] # Lista de comprensión
print(archivos) # Salida: ['datos.csv', 'script.py', ...]
```

2. Interacción con el sistema operativo:

- Un programa puede "entender" el sistema operativo, CPU, memoria o red.
- **Ejemplo:**

```
import platform

# "Percibir" el sistema operativo
sistema = platform.system()
print(sistema) # Salida: Windows, Linux, Darwin (macOS)
```


3. Lectura de datos del entorno:

- En la adquisición de datos, un programa "percibe" datos de archivos, sensores o APIs.
- **Ejemplo:**

```
import csv

# "Percibir" datos de un archivo CSV
with open('datos.csv', 'r') as file:
    datos = [row for row in csv.reader(file)] # Lista de comprensión
print(datos) # Salida: [['Temperatura', 'Humedad'], ['20', '60'], ...]
```

2.2. Ventajas y Limitaciones

Ventajas:

- **Adaptabilidad:** Permite que un programa reaccione a cambios en el entorno (por ejemplo, nuevos archivos o errores).
- **Robustez:** Manejar excepciones y validar datos mejora la fiabilidad.
- **Eficiencia:** "Percibir" el entorno optimiza el uso de recursos (por ejemplo, elegir el formato de datos adecuado).
- **Contexto:** Facilita la integración con sistemas externos (bases de datos, APIs).

Limitaciones:

- **Complejidad:** Implementar consciencia del entorno requiere manejar múltiples casos (excepciones, formatos).
- **Dependencia del entorno:** Los programas pueden fallar si el entorno no está configurado correctamente.
- **Overhead:** Monitorear recursos o validar datos puede consumir tiempo.

2.3. Comparación con Otras Etapas

- **Adquisición de datos vs. Consciencia del entorno:** La adquisición recopila datos; la consciencia del entorno implica "entender" de dónde vienen y cómo usarlos.
- **Representación de datos vs. Consciencia del entorno:** La consciencia del entorno guía cómo se estructuran o presentan los datos (por ejemplo, elegir CSV o tabla).
- **Outliers vs. Consciencia del entorno:** Detectar outliers requiere "percibir" la distribución de los datos en el contexto del entorno.

2.4. Otros Significados de Consciencia del Entorno

Si tu pregunta apuntaba a otro contexto, aquí hay interpretaciones alternativas relevantes:

1. Robótica e IA:

- La consciencia del entorno es la capacidad de un sistema (como un robot) para percibir su entorno físico mediante sensores (luz, sonido, distancia) y tomar decisiones.
- Ejemplo: Un robot aspirador usa sensores para "entender" la ubicación de obstáculos, similar a la adquisición de datos.
- Relación: Los datos adquiridos se representan en estructuras como listas para su procesamiento.

2. Ecología y sostenibilidad:

- La consciencia del entorno se refiere a entender el impacto ambiental de las actividades computacionales (por ejemplo, consumo energético de scripts).
- Ejemplo: Monitorear el uso de CPU para optimizar un programa de análisis de datos.
- Relación: Los entornos virtuales pueden configurarse para usar bibliotecas eficientes.

¿Para que sirve un algoritmo de apoyo a la toma de decisiones?

1. Introducción

Un **algoritmo de apoyo a la toma de decisiones** es un procedimiento computacional diseñado para **asistir a personas o sistemas en la elección de la mejor opción** entre varias alternativas, basándose en datos, criterios predefinidos y objetivos específicos. Estos algoritmos procesan información, evalúan opciones y generan recomendaciones o decisiones óptimas, reduciendo la subjetividad y mejorando la eficiencia en la toma de decisiones.

En el contexto del tratamiento de datos, los algoritmos de apoyo a la toma de decisiones son esenciales para:

- Analizar datos adquiridos (por ejemplo, de sensores o bases de datos).
- Identificar patrones, anomalías (como outliers) o tendencias.
- Proporcionar recomendaciones basadas en reglas, modelos estadísticos o lógica.

Estos algoritmos se utilizan en campos como la ciencia de datos, la ingeniería, los negocios, la medicina y la robótica, y se integran con conceptos como la **consciencia del entorno** (para "percibir" datos relevantes) y la **representación de datos** (para estructurar la información).

2. Desarrollo

2.1. ¿Para qué sirve?

Un algoritmo de apoyo a la toma de decisiones sirve para:

1. **Automatizar decisiones:** Ejecutar elecciones basadas en reglas predefinidas, reduciendo la intervención humana.
 - Ejemplo: Decidir si encender un sistema de riego según la humedad del suelo.
2. **Optimizar resultados:** Seleccionar la mejor opción según criterios como costo, tiempo o calidad.
 - Ejemplo: Elegir la ruta más corta para un vehículo de entrega.
3. **Reducir sesgos:** Basar decisiones en datos objetivos en lugar de intuiciones subjetivas.
 - Ejemplo: Evaluar solicitudes de crédito según métricas financieras.
4. **Manejar complejidad:** Procesar grandes volúmenes de datos o múltiples variables que serían difíciles de analizar manualmente.
 - Ejemplo: Diagnosticar enfermedades basándose en síntomas y pruebas médicas.
5. **Mejorar eficiencia:** Acelerar el proceso de decisión en entornos críticos.
 - Ejemplo: Detectar fraudes en transacciones en tiempo real.

2.2. Tipos de Algoritmos de Apoyo a la Toma de Decisiones

Existen varios enfoques para diseñar estos algoritmos, dependiendo del problema y los datos disponibles:

1. Basados en reglas:

- Usan condiciones lógicas (if-then-else) para tomar decisiones.
- **Ejemplo:**

```
# Decidir si activar un ventilador según la temperatura
temperatura = 30
if temperatura > 25:
    decision = "Encender ventilador"
else:
    decision = "Apagar ventilador"
print(decision) # Salida: Encender ventilador
```

2. Basados en puntajes o pesos:

- Asignan puntajes a las opciones según criterios y eligen la de mayor puntaje.
- **Ejemplo:**

```
# Elegir el mejor proveedor según costo y calidad
proveedores = [
    {'nombre': 'A', 'costo': 100, 'calidad': 8},
    {'nombre': 'B', 'costo': 120, 'calidad': 9},
    {'nombre': 'C', 'costo': 80, 'calidad': 6}
]

# Calcular puntaje (menor costo y mayor calidad)
puntajes = [(p['nombre'], (1000 / p['costo']) + p['calidad']) for p in proveedores]
mejor_proveedor = max(puntajes, key=lambda x: x[1])[0]
print(f"Mejor proveedor: {mejor_proveedor}") # Salida: Proveedor A
```

2.3. Ventajas y Limitaciones

Ventajas:

- **Automatización:** Reduce la carga de decisiones manuales.
- **Objetividad:** Minimiza sesgos humanos al basarse en datos.
- **Escalabilidad:** Puede manejar grandes volúmenes de datos o decisiones complejas.
- **Consistencia:** Aplica criterios uniformes en todas las decisiones.

Limitaciones:

- **Dependencia de datos:** Requiere datos de calidad (sin errores ni outliers).
- **Complejidad:** Diseñar algoritmos robustos puede ser desafiante.
- **Limitaciones contextuales:** Puede no captar matices humanos o situaciones imprevistas.

¿Qué es la inteligencia artificial?

1. Introducción

La **inteligencia artificial (IA)** es un campo de la informática que busca desarrollar sistemas y máquinas capaces de realizar tareas que normalmente requieren **inteligencia humana**, como razonamiento, aprendizaje, percepción, toma de decisiones, y comprensión del lenguaje. La IA permite a las computadoras procesar datos, identificar patrones, y tomar decisiones o generar resultados basados en esos datos, a menudo imitando capacidades humanas pero con mayor velocidad y escala.

En el contexto del tratamiento de datos, la IA se apoya en:

- **Adquisición de datos** para obtener información del entorno.
- **Representación de datos** para estructurar la información procesable.
- **Algoritmos** (como los de apoyo a la toma de decisiones) para generar resultados.
- **Consciencia del entorno** para interactuar dinámicamente con el contexto.

La IA abarca desde sistemas simples basados en reglas hasta modelos avanzados que aprenden de grandes volúmenes de datos, y es fundamental en aplicaciones modernas como asistentes virtuales, vehículos autónomos, y análisis de datos.

2. Desarrollo

2.1. Definición y Componentes

La IA se define como la capacidad de un sistema para:

- **Percibir:** Captar información del entorno (por ejemplo, datos de sensores o texto).
- **Razonar:** Procesar datos para inferir conclusiones o resolver problemas.
- **Actuar:** Tomar decisiones o ejecutar acciones basadas en el razonamiento.
- **Aprender:** Mejorar su comportamiento con la experiencia (en el caso de la IA basada en aprendizaje).

Los componentes clave de la IA incluyen:

1. **Datos:** La base para entrenar o alimentar sistemas de IA (relacionado con la adquisición de datos).
2. **Algoritmos:** Reglas o modelos que procesan los datos (por ejemplo, algoritmos de toma de decisiones).
3. **Hardware:** Computadoras o dispositivos especializados (como GPUs, aunque no los detallaré aquí).
4. **Software:** Programas que implementan la IA, a menudo en entornos virtuales.

2.2. Tipos de Inteligencia Artificial

La IA se clasifica según su capacidad y complejidad:

1. IA débil (Narrow AI):

- Diseñada para tareas específicas, como reconocimiento de voz o recomendación de productos.
- Ejemplo: Un sistema que clasifica correos como spam o no spam.
- **Ejemplo en Python** (clasificación simple basada en reglas):

```
correos = [
    {'texto': 'Oferta especial', 'es_spam': True},
    {'texto': 'Reunión mañana', 'es_spam': False}
]
# Clasificar correos
clasificados = [c['texto'] for c in correos if c['es_spam']]
print(clasificados) # Salida: ['Oferta especial']
```

2. IA general (General AI):

- Capaz de realizar cualquier tarea intelectual humana, pero aún es teórica y no existe actualmente.
- Ejemplo: Una IA que puede aprender cualquier profesión.

3. IA fuerte (Superintelligent AI):

- Superaría la inteligencia humana en todos los aspectos, pero es especulativa.

La mayoría de las aplicaciones actuales son de **IA débil**, que se especializa en tareas concretas.

2.3. ¿Para qué sirve la IA?

La IA tiene múltiples aplicaciones, conectadas con tus temas:

1. Automatización:

- Automatiza tareas repetitivas, como clasificar datos adquiridos.
- **Ejemplo:**

```
import csv
with open('ventas.csv', 'r') as file:
    ventas = [float(row[0]) for row in csv.reader(file)]
categorias = ['Alta' if v > 1000 else 'Baja' for v in ventas]
print(categorias) # Salida: ['Baja', 'Alta', ...]
```

2. Toma de decisiones:

- Mejora los algoritmos de apoyo a la toma de decisiones al aprender de datos.
- **Ejemplo:**

```
clientes = [  
    {'ingresos': 5000, 'deudas': 1000},  
    {'ingresos': 2000, 'deudas': 1500}  
]  
decisiones = ['Aprobar' if c['ingresos'] > 2 * c['deudas'] else 'Rechazar' for c in clientes]  
print(decisiones) # Salida: ['Aprobar', 'Rechazar']
```

3. Detección de anomalías:

- Identifica outliers en datasets.
- **Ejemplo:**

```
datos = [20, 22, 19, 25, 100]  
sorted_datos = sorted(datos)  
Q1 = sorted_datos[len(datos)//4]  
Q3 = sorted_datos[3*len(datos)//4]  
IQR = Q3 - Q1  
anomalias = [x for x in datos if x < Q1 - 1.5 * IQR or x > Q3 + 1.5 * IQR]  
print(anomalias) # Salida: [100]
```

2.4. Ventajas y Limitaciones

Ventajas:

- **Eficiencia:** Procesa grandes volúmenes de datos rápidamente.
- **Automatización:** Libera a los humanos de tareas repetitivas.
- **Precisión:** Mejora decisiones con datos objetivos.
- **Adaptabilidad:** Aprende y se ajusta a nuevos datos (en IA basada en aprendizaje).

Limitaciones:

- **Dependencia de datos:** Requiere datos de calidad, sin errores ni outliers.
- **Complejidad:** Diseñar sistemas de IA puede ser costoso y técnico.
- **Ética:** Puede generar sesgos si los datos son sesgados.
- **Limitaciones contextuales:** La IA débil no comprende matices humanos.

2.7. Comparación con Otras Etapas

- **Adquisición de datos vs. IA:** La adquisición proporciona los datos; la IA los procesa para generar resultados.
- **Representación de datos vs. IA:** La representación estructura los datos; la IA los usa para aprender o decidir.
- **Outliers vs. IA:** La IA detecta outliers como parte de su análisis.
- **Consciencia del entorno vs. IA:** La IA "percibe" el entorno para actuar, como en robótica.
- **Algoritmos de toma de decisiones vs. IA:** La IA generaliza estos algoritmos con aprendizaje o modelos avanzados.

¿En que consiste el proceso de toma de decisiones?

1. Introducción

El **proceso de toma de decisiones** es una secuencia estructurada de pasos que una persona, grupo o sistema sigue para **seleccionar la mejor opción** entre varias alternativas, con el objetivo de resolver un problema, alcanzar una meta o responder a una situación específica. Este proceso implica identificar el problema, recopilar información, evaluar opciones y elegir una acción, considerando datos, criterios y restricciones.

En el contexto del tratamiento de datos, la toma de decisiones se apoya en:

- **Adquisición de datos** para obtener información relevante.
- **Representación de datos** para organizar la información de forma comprensible.
- **Consciencia del entorno** para entender el contexto (por ejemplo, datos disponibles o recursos).
- **Algoritmos de apoyo a la toma de decisiones** para automatizar o guiar el proceso.
- **Inteligencia artificial** para mejorar la toma de decisiones con aprendizaje o análisis avanzado.

El proceso puede ser humano, automatizado (por ejemplo, mediante un script en Python), o híbrido, y es fundamental en campos como la ciencia de datos, los negocios, la ingeniería y la vida cotidiana.

2. Desarrollo

2.1. Etapas del Proceso de Toma de Decisiones

El proceso de toma de decisiones generalmente sigue estas etapas, que se detallan con ejemplos relevantes:

1. Identificar el problema o la necesidad:

- Reconocer que existe una situación que requiere una decisión.
- Ejemplo: Un sistema detecta que la temperatura de una máquina excede un umbral seguro.
- **Ejemplo en Python:**

```
temperatura = 85 # Dato adquirido
problema = "Temperatura alta" if temperatura > 80 else "Sin problema"
print(problema) # Salida: Temperatura alta
```

2. Recopilar información relevante:

- Obtener datos necesarios para entender el problema y las opciones disponibles (relacionado con la adquisición de datos).
- Ejemplo: Consultar lecturas históricas de temperatura o parámetros de la máquina.
- **Ejemplo en Python:**

```
import csv
# "Percibir" datos históricos
with open('temperaturas.csv', 'r') as file:
    datos = [float(row[0]) for row in csv.reader(file)] # Lista de comprensión
print(datos) # Salida: [75, 80, 85, 90]
```

2.2. Factores que Influyen

- **Calidad de los datos:** Datos con outliers o errores pueden llevar a decisiones incorrectas.
- **Criterios:** Claridad en los objetivos (por ejemplo, minimizar costos o maximizar seguridad).
- **Recursos:** Disponibilidad de herramientas y tiempo.
- **Contexto:** El entorno (por ejemplo, urgencia o restricciones) afecta las prioridades.

2.3. Ventajas y Limitaciones

Ventajas:

- **Estructura:** Proporciona un enfoque sistemático para resolver problemas.
- **Objetividad:** Basarse en datos reduce sesgos (si los datos son confiables).
- **Flexibilidad:** Aplicable a decisiones simples y complejas.
- **Mejora continua:** La evaluación de resultados permite ajustar futuras decisiones.

Limitaciones:

- **Dependencia de datos:** Sin información adecuada, las decisiones pueden fallar.
- **Subjetividad:** Los criterios o prioridades pueden variar entre personas.
- **Tiempo:** Recopilar y evaluar datos puede ser lento en situaciones urgentes.
- **Complejidad:** Decisiones con muchas variables requieren herramientas avanzadas.

2.4. Comparación con Otros Conceptos

- **Adquisición de datos vs. Toma de decisiones:** La adquisición proporciona los datos; la toma de decisiones los usa para actuar.
- **Representación de datos vs. Toma de decisiones:** La representación organiza los datos; la toma de decisiones los interpreta.
- **Outliers vs. Toma de decisiones:** Los outliers deben manejarse para evitar decisiones erróneas.
- **Consciencia del entorno vs. Toma de decisiones:** La consciencia del entorno informa el contexto para decidir.
- **Algoritmos de apoyo vs. Toma de decisiones:** Los algoritmos automatizan partes del proceso.
- **IA vs. Toma de decisiones:** La IA optimiza el proceso con aprendizaje o modelos avanzados.

¿En que consiste el preprocesamiento de datos?

1. Introducción

El **preprocesamiento de datos** es el conjunto de técnicas y procesos aplicados para **limpiar, transformar y organizar datos crudos** obtenidos durante la adquisición de datos, con el objetivo de prepararlos para su análisis, modelado o uso en sistemas como algoritmos de toma de decisiones o inteligencia artificial. Este paso es crucial en el tratamiento de datos, ya que los datos crudos suelen contener errores, valores faltantes, inconsistencias o formatos inadecuados que pueden afectar la calidad de los resultados.

El preprocesamiento asegura que los datos sean:

- **Limpios:** Sin errores, duplicados o outliers.
- **Consistentes:** En formatos uniformes y relevantes.
- **Relevantes:** Incluyen solo la información necesaria para el análisis.

En el contexto de tus temas previos:

- **Adquisición de datos:** El preprocesamiento transforma los datos crudos adquiridos (por ejemplo, de sensores o APIs).
- **Representación de datos:** Convierte los datos en estructuras adecuadas (listas, tablas).
- **Outliers:** El preprocesamiento incluye detectar y manejar valores atípicos.
- **Consciencia del entorno:** Requiere entender el contexto de los datos (fuentes, formatos).
- **Algoritmos de toma de decisiones e IA:** Los datos preprocesados son esenciales para que estos sistemas generen resultados precisos.
- **Listas de comprensión:** Facilitan tareas de limpieza y transformación.
- **Entornos:** Proveen las herramientas necesarias para el preprocesamiento.

2. Desarrollo

2.1. Etapas del Preprocesamiento de Datos

El preprocesamiento de datos consta de varias etapas, que se detallan con ejemplos en Python estándar o con Pandas, evitando Matplotlib, NumPy y eval:

1. Limpieza de datos:

- Eliminar o corregir errores, duplicados, valores faltantes o inconsistencias.
- **Ejemplo** (manejo de valores faltantes)

```
import csv
# "Percibir" datos históricos
with open('temperaturas.csv', 'r') as file:
    datos = [float(row[0]) for row in csv.reader(file)] # Lista de comprensión
print(datos) # Salida: [75, 80, 85, 90]

datos = [20, None, 22, 19, None, 25]
# Reemplazar None con el valor anterior
datos_limpios = []
ultimo_valor = 0
for x in datos:
    if x is None:
        datos_limpios.append(ultimo_valor)
    else:
        datos_limpios.append(x)
        ultimo_valor = x
print(datos_limpios) # Salida: [20, 20, 22, 19, 19, 25]
```

2. Manejo de valores atípicos (outliers):

- Detectar y tratar outliers para evitar sesgos en el análisis (relacionado con tu consulta sobre outliers).
- **Ejemplo** (filtrar outliers con IQR):

```
datos = [20, 22, 19, 25, 100, 21, 23]
sorted_datos = sorted(datos)
Q1 = sorted_datos[len(datos)//4]
Q3 = sorted_datos[3*len(datos)//4]
IQR = Q3 - Q1
datos_limpios = [x for x in datos if Q1 - 1.5 * IQR <= x <= Q3 + 1.5 * IQR]
print(datos_limpios) # Salida: [20, 22, 19, 25, 21, 23]
```

3. Normalización y estandarización:

- Ajustar los datos a un rango común (normalización) o a una distribución estándar (estandarización) para compararlos.
- **Ejemplo** (normalización min-max con lista de comprensión):

```
datos = [20, 22, 19, 25]
min_val = min(datos)
max_val = max(datos)
datos_normalizados = [(x - min_val) / (max_val - min_val) for x in datos]
print(datos_normalizados) # Salida: [0.16666666666666666, 0.5, 0.0, 1.0]
```

4. Transformación de datos:

- Convertir datos a formatos adecuados, como cambiar tipos, codificar categorías o agrupar valores.
- **Ejemplo** (convertir cadenas a números):

```
datos = ['20', '22', '19', '25']
datos_transformados = [float(x) for x in datos]
print(datos_transformados) # Salida: [20.0, 22.0, 19.0, 25.0]
```

2.3. Ventajas y Limitaciones

Ventajas:

- **Calidad de datos:** Mejora la precisión de los análisis al eliminar errores y outliers.
- **Eficiencia:** Prepara datos para un procesamiento más rápido.
- **Consistencia:** Uniformiza los datos para su uso en algoritmos o IA.
- **Relevancia:** Reduce el ruido al eliminar datos irrelevantes.

Limitaciones:

- **Pérdida de información:** Filtrar datos (como outliers válidos) puede omitir información importante.
- **Tiempo:** El preprocesamiento puede ser laborioso para datasets grandes.
- **Subjetividad:** Las decisiones sobre cómo manejar valores faltantes o outliers dependen del contexto.
- **Dependencia de herramientas:** Aunque usamos Python estándar, datasets complejos requieren bibliotecas avanzadas.

2.4. Comparación con Otros Conceptos

- **Adquisición de datos vs. Preprocesamiento:** La adquisición recopila datos crudos; el preprocesamiento los limpia y transforma.
- **Representación de datos vs. Preprocesamiento:** El preprocesamiento prepara los datos para su representación (por ejemplo, en listas o tablas).
- **Outliers vs. Preprocesamiento:** El manejo de outliers es una etapa clave del preprocesamiento.
- **Consciencia del entorno vs. Preprocesamiento:** La consciencia del entorno guía las decisiones sobre cómo preprocesar (por ejemplo, detectar formatos).
- **Algoritmos de toma de decisiones/IA vs. Preprocesamiento:** El preprocesamiento asegura datos de calidad para estos sistemas.
- **Proceso de toma de decisiones vs. Preprocesamiento:** El preprocesamiento proporciona datos fiables para identificar problemas y evaluar opciones.

Fuentes consultadas

Documentación oficial de Python:

Python Software Foundation. (2023). "Data Structures: List Comprehensions". Disponible en: <https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions>

Documentación oficial de Matplotlib:

Matplotlib Development Team. (2023). "Matplotlib: Visualization with Python". Disponible en: <https://matplotlib.org/stable/>

Incluye tutoriales, ejemplos y referencia de la API.

Documentación oficial de NumPy:

NumPy Development Team. (2023). "NumPy Documentation". Disponible en: <https://numpy.org/doc/stable/>

Incluye guías de usuario, referencia de la API, y ejemplos.

Documentación oficial de Python:

Python Software Foundation. (2023). "Built-in Functions: eval". Disponible en: <https://docs.python.org/3/library/functions.html#eval>

Explicación oficial de eval y sus parámetros.

Libro: "Python for Data Analysis":

McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.

Cubre la adquisición de datos desde archivos y APIs, con ejemplos en Pandas.

Libro: "Python for BeautifulSoup Data Analysis":

McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.

Cubre representación de datos con Pandas y NumPy.

Libro: "Python for Data Analysis":

McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.

Cubre representación de datos con Pandas y estructuras básicas.

Libro: "Artificial Intelligence: A Guide to Intelligent Systems":

Negnevitsky, M. (2011). *Artificial Intelligence: A Guide to Intelligent Systems*. Pearson.

Introducción a los fundamentos de la IA.